

CALIBER: BUILDING A GOVERNED CONTROL PLANE FOR AI-AGENT RESOURCES, WORKFLOWS, AND PROGRESSIVE AUTONOMY

Anonymous Authors¹

ABSTRACT

Production AI-agent systems rarely fail because a prompt alone regressed; they fail across prompts, workflows, tools, MCP servers, and retrieval corpora whose release processes are usually fragmented. We present CALIBER, a layered control plane that governs those heterogeneous resources through typed records, evidence-aware release workflows, explicit per-family guarantees, and one shared operational surface spanning the browser UI, a served management API, a remote Python SDK and CLI, and a server-side plugin contract. The current implementation reuses MLflow as substrate rather than rebuilding observability and prompt-registry primitives, and packages a single-origin enterprise surface with workflow publication, review queues, gateway controls, release signoff, and a permissioned copilot loop. We report the design decisions and operational lessons from building the system, including where the architecture deliberately stops: guarantees remain family-specific, assistant autonomy is scoped, and distributed atomicity across external systems is replaced by durable intent and reconciliation. Large-scale latency and operator-study results remain future work; the evidence in this paper is executable structural and implementation evidence, not a performance claim.

1 INTRODUCTION

Enterprise teams now operate agentic systems whose behavior depends on far more than a single prompt. A production incident may be caused by a prompt, a workflow branch, an MCP server policy, a tool implementation, a retrieval corpus refresh, or an evaluation asset that silently drifted. Existing platforms partially solve this for prompts by adding versions, mutable aliases, and rollback, but the broader release surface of an AI-agent application remains fragmented across dashboards, source files, and operational runbooks (MLflow Project, 2026b; LangChain, Inc., 2026; Langfuse GmbH, 2026).

CALIBER addresses that operational gap as a control plane rather than as a new composition framework. Agents and workflows may be authored elsewhere; CALIBER governs the resources they depend on and the release path that changes them. The design centers on three practical requirements:

1. one governable record shape for heterogeneous AI-agent resources;
2. evidence and evaluation that can block advancement

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the Machine Learning and Systems (MLSys) Conference. Do not distribute.

rather than merely annotate it; and

3. an operational surface that remains usable by browser operators, automation, SDK consumers, and extension authors without forcing each group onto a different private interface.

This paper is an MLSys Industry Track adaptation of the full CALIBER technical manuscript. The focus here is the industrial system problem, the implemented architecture, the engineering tradeoffs, and the operational lessons. Our contributions are:

- a layered control-plane architecture for governed AI-agent resources and workflows;
- a practical account of per-family guarantees rather than a false universal release contract;
- a shared operational surface spanning UI, API, SDK, CLI, and plugin seams; and
- implementation and structural evidence from a live, repository-grounded system.

2 INDUSTRIAL PROBLEM AND DESIGN TARGET

The core problem is not lack of observability. Production systems can already trace requests, store prompts, and attach

evaluation results (MLflow Project, 2026a). The missing piece is a *governed release path* across the non-code resources that actually determine agent behavior. In practice, teams need to answer questions such as: Which workflow definition is live? Which knowledge-base build answered this incident? Which gate failed, and who overrode it? Can an operator roll back safely without redeploying the application?

Those questions are harder than prompt versioning because the artifact families are genuinely heterogeneous. A prompt may have a live alias; a test set is evidence and has no live target; a judge is evaluated for agreement but is not itself released. That heterogeneity drove the most important design choice in CALIBER: the platform uses one shared governed-record abstraction, but the release, rollback, evaluation, and evidence semantics remain explicit per family.

3 SYSTEM OVERVIEW

CALIBER models each managed resource as a typed governed record with identity, version history, authority, and audit. Families currently span prompts, tools, skills, MCP servers, workflows, knowledge bases, test sets, judges, and agent anchors. Around that record shape, the platform organizes six lifecycle modes: author, test, evaluate, calibrate, release, and observe.

The runtime system is one Starlette-based ASGI application plus a React SPA served from the same origin. Operationally, this matters because the browser UI, the management API, and published workflow services share one deployment boundary, one authentication model, and one audit substrate rather than competing side channels. The current implementation exposes 490 route declarations across 48 modules, and the route table is mirrored into a served OpenAPI document, a lightweight remote Python SDK, and a thin CLI.

4 ARCHITECTURE AND DESIGN DECISIONS

4.1 Layering and family-specific guarantees

The architectural claim is deliberately narrow: adjacency in a layered architecture confers *capability availability*, not *capability inheritance*. A family placed in the governed-asset layer can use shared release, evidence, and audit primitives, but it gains those guarantees only if it explicitly wires them.

This is not rhetorical caution; it is the only honest contract across heterogeneous families. A prompt can rotate a live alias. A workflow can publish a service version or restore a checkpointed deployment target. A test set has no live target to rotate. Treating these as one uniform release theorem would either erase the differences or exclude useful families from the platform.

The main engineering tradeoffs are practical, not ornamental. We reused MLflow because prompt-registry, tracing, and gateway primitives already exist, accepting the dual-write boundaries that remain between SQL state and external effects. We kept guarantees family-specific because heterogeneous assets do not share one meaningful release contract, accepting the burden of publishing the guarantee surface explicitly. We chose a single-origin control plane so the UI, API, workflow services, and docs share one auth and audit boundary, accepting that published-service traffic and operator traffic compete inside one process model. We split the remote SDK from the plugin SDK so automation and extension authors do not need the server runtime just to call a route or declare a plugin, accepting the cost of additional package surfaces. And we kept assistant autonomy scoped, because delegated draft review is useful operationally but is not evidence that open-ended autonomous release is safe.

4.2 Intent-first release and reconciliation

The platform’s strongest release guarantee today is on the prompt family. Every governed prompt promotion first writes a durable release intent that records the outgoing target, the incoming version, the acting principal, and the evidence reference. Only then does the system perform the external alias move. If the external system changes and the local settlement does not, reconciliation can inspect the durable intent and converge it. This is not distributed atomicity; it is explicit, observable compensation.

4.3 Broker-free execution model

Background work is drained by 9 in-process loops. Claim-and-lease loops use relational ownership rather than a dedicated broker. This keeps deployment lightweight and makes SQL the authoritative arbitration point, which suits an operator-facing control plane. The trade is explicit: queue capacity and request capacity share a process model, and per-process limiters remain per-process until lifted into a shared store.

5 INTERFACES AND EXTENSIBILITY

5.1 One management surface, multiple consumers

The management surface is deliberately shared. The browser uses the same governed-asset API shape the remote SDK consumes. The CLI sits on top of the SDK rather than on private admin calls. A machine-generated OpenAPI document exposes the current route table, which keeps typed clients and documentation aligned to the running surface rather than to hand-maintained wrappers.

The current repository also separates extension and automation concerns into three peer packages:

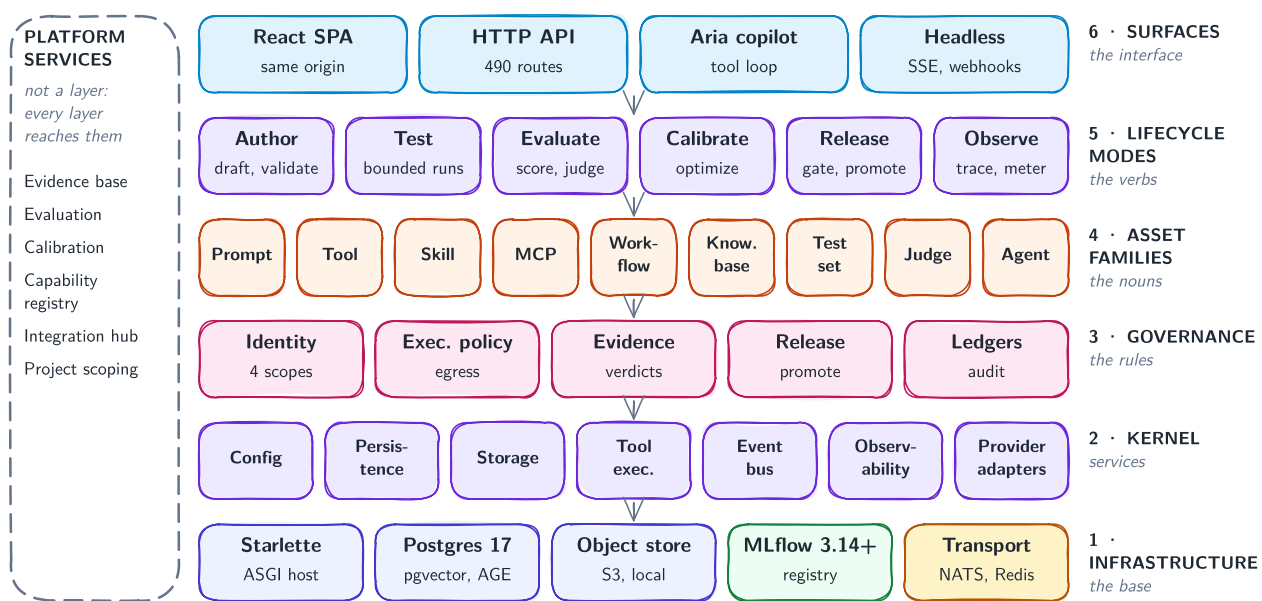


Figure 1. CALIBER is organized as a layered control plane. Shared substrate and governance capabilities sit beneath heterogeneous asset families and lifecycle modes; surfaces above them expose one operational system to browser users, automation, and extension authors.

- `caliber-sdk`: lightweight remote Python client for the management API;
- `caliber-plugin-sdk`: server-side extension contract kept free of heavy runtime dependencies;
- `caliber-cli` / `caliberctl`: non-interactive operator automation over the SDK.

That split matters in practice. A developer script should not need to install MLflow, SQLAlchemy, and the server runtime merely to call a route. Conversely, a plugin contract should not silently depend on a live database or request context.

5.2 Operational capability surfaces

Several recent additions make the platform more than a prompt or workflow registry. The LLM Gateway is exposed as a governed surface for endpoint discovery, guardrail attachment, pricing, and trace-derived usage. Release sign-off adds typed candidates, weighted criteria, waivers, immutable go/no-go decisions, and durable report jobs that bind Allure-compatible evidence to a candidate snapshot. The workflow catalog ships sixteen installable cookbook examples that create paused drafts rather than live deployments, which turns system exemplars into governed artifacts rather than documentation alone.

6 DEPLOYMENT AND CURRENT EVIDENCE

The current implementation spans 327 Python modules (129k lines), 293 TypeScript modules (177k lines), 86 database migrations, 31 workflow node types, and 29 hand-written assistant tools plus 7 registry-projected capabilities. The development topology bundles PostgreSQL with pgvector and Apache AGE, object storage, MLflow, gateway support, and optional event backends. The externalized topology preserves the same application surface while swapping the infrastructure under it.

The executable evidence today is structural and implementation evidence, not a throughput claim. Concretely:

- deterministic fakes sit behind provider seams, so the control plane is testable without paid providers;
- deterministic structural checks verify queue ownership, release intent ordering, reconciliation, resolver fallback, and publication gating;
- browser, backend, docs, packaging, and cookbook journeys are all part of the CI contract; and
- generated counts, diagrams, and documentation are derived from the repository rather than manually transcribed.

This evidence boundary is deliberate. We do *not* claim that current results establish latency, throughput, large-scale

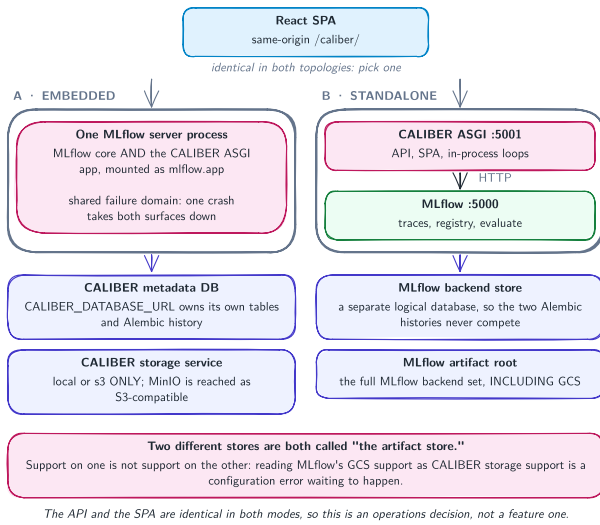


Figure 2. Two supported topologies: a loopback-oriented bundled stack for development and an externalized deployment using managed infrastructure. The functional surface is the same in both cases.

availability, or operator productivity. Those remain measurement obligations.

7 LESSONS LEARNED

The real problem is cross-family release semantics. Prompt lifecycle management is no longer novel by itself. The harder engineering problem is extending governed release discipline to workflows, tools, MCP servers, knowledge versions, and review assets without pretending they all behave like prompts.

Shared UI is not shared semantics. The most likely operator mistake is to see one version-history component rendered in several places and infer one rollback guarantee. In practice, rollback means alias restore for prompts, checkpoint restoration for workflows, activation-history derivation for knowledge bases, and “not supported” for some families. The system must publish that difference explicitly.

A durable intent is often the practical alternative to atomicity. External registries and object stores do not join the caller’s SQL transaction. The operationally useful property is therefore not impossible atomicity but a durable record of intended change plus explicit reconciliation and abandonment surfaces.

SDK boundaries are architecture, not packaging polish. Separating the remote SDK from the plugin SDK forced

clearer contracts. The same decision that keeps developer automation lightweight also prevents server-only assumptions from leaking into third-party extension points.

8 LIMITATIONS AND FUTURE WORK

Three limitations are central.

First, the main quantitative evaluation remains unrun. This paper reports a built system and executable structural evidence, not a completed scale or latency study.

Second, guarantees remain per family. The platform publishes a checked capability surface, but it does not yet mechanically prove that every declared capability is implemented by every handler that claims it.

Third, progressive autonomy is scoped. Assistant-driven draft review and delegated publication exist, but they are not evidence that open-ended autonomous operations are production-safe. The enforced human decision in the refinement path remains a deliberate design boundary.

The next highest-value engineering steps are straightforward to state even if not small to implement: execute the evaluation protocol; add multi-environment promotion; complete capability-to-handler enforcement; and introduce a true OS-enforced isolation backend for untrusted tool execution.

9 CONCLUSION

CALIBER is an implemented answer to a practical enterprise problem: how to govern the non-code resources that make AI-agent systems behave the way they do. Its central result is not that all such resources can share one universal release contract, but that they can share one control-plane architecture while keeping their guarantees explicitly family-specific. The engineering consequence is a system whose UI, API, SDK, CLI, and plugin seams are aligned around one governed surface, and whose release path records enough evidence and authority to make changes reviewable after the fact. That is a narrower claim than “fully autonomous agent governance,” but it is the one the current system actually supports.

REFERENCES

- LangChain, Inc. Manage prompts in LangSmith. <https://docs.langchain.com/langsmith/manage-prompts>, 2026. Documents commit-based version history, custom commit tags, the reserved staging and production tags, `client.pull_prompt("name:production")` resolution, rollback to a previous commit, and owners-only tag permissions. Accessed 2026-08-03.

Langfuse GmbH. Prompt version control. <https://langfuse.com/docs/prompt-management/features/prompt-version-control>, 2026. Documents version identifiers, the production and latest labels plus custom labels, client-side resolution by label at fetch time, rollback by re-assigning the production label, and protected labels restricted by role. Accessed 2026-08-03.

MLflow Project. MLflow for GenAI: Tracing, evaluation, and assessments. <https://mlflow.org/docs/latest/genai/>, 2026a. The current-capability source for claims about MLflow traces, assessments, scorers, and evaluation entry points. The 2018 paper predates all of these and is cited only for MLflow's original contribution. Accessed 2026-08-03.

MLflow Project. MLflow prompt registry. <https://mlflow.org/docs/latest/genai/prompt-registry/>, 2026b. Documents commit-based prompt versioning, mutable aliases, prompt- and version-level tags, commit messages, and immutable versions. Accessed 2026-08-03.